Aim : To learn and implement the MERN stack based on the provided documentation and convert one of the previous React and Node.js projects into a MERN application

Design :

Algorithm :

1. Start the application.
2. Initialize the state in the client-side React component, including an array called todos to store tasks and a string called newTodo to store the text of the new task.
3. In the componentDidMount lifecycle method, send a GET request to the server to fetch existing tasks from the MongoDB database.
4. Handle the response from the server by updating the todos state with the retrieved data.
5. Create two methods, handleInputChange and handleSubmit, to handle user input for adding new tasks.
6. In the handleInputChange method, update the newTodo state as the user types.
7. In the handleSubmit method, check if the input is empty and return if so.
8. If the input is not empty, create a new task object with the task and completed properties.
9. Send a POST request to the server to add the new task to the database.
10. Update the state with the newly added task and reset the newTodo input field.
11. Render the UI, displaying the task list, input field, and a button to add tasks.
12. Map through the todos array to display each task in the list.

Frontend Components (React) :

1. App.js component for the main application.
2. Task List component to display the list of tasks.
3. Task Item component to represent individual tasks.

4. Task Form component for adding new tasks.

Frontend Functionality (React) :

1. Fetching tasks from the backend.

2. Adding a new task to the list.

3. Handling user input for tasks.

Backend (Node.js with Express):

1. Defining API endpoints for fetching and adding tasks.

2. Connecting to the MongoDB database.

Database (MongoDB) :

1. Creating a schema for tasks and a model.

2. Storing task data with the Todo schema.

3. Performing CRUD operations for tasks.